COLORADO CONNECTION REPEATERS
"Connecting Colorado's Amateur Community"

Rocky Mountain Ham Radio
Teach ... Learn ... Operate ... Support. Be part of the team.

Voter2-2K
Users Guide

EDITED BY: WILL BEALS

CONTRIBUTORS: BEN MATTHEWS, DAVE MACIOROWSKI, DOUG SHARP, JAMES CIZEK, JEFF CARRIER, JOHN MAXWELL, MARK SKELTON

DRAFT VERSION: 0.09

NOVEMBER 1, 2026

An Open-Source Hardware and Software Project
www.colcon.org
www.rmham.org

1 INTRODUCTION 6

2 RELEASE STATE 7

3 OTHER DOCUMENTS 7

4 COMMUNICATION INTERFACES 8

4.1 10MHz Reference 8

4.2 USB Console 8

4.3 ETHERNET 8
4.3.1 ALLSTARLINK Port 9
!Unexpected End of Formula
4.3.3 Logger Port 9
4.3.4 Ser2Net (RFC2217) Port 9
4.3.5 TFTP Port(s) 10

4.4 MTR UART 10

4.5 LEDs 10

4.6 GPS(es) 12
4.6.1 Ublox M8n 12
4.6.2 ST Teseo LIV4F 12

4.7 MTR-2000 Backplane 12
4.7.1 Rx Audio 12
4.7.2 Tx Audio 12
4.7.3 RSSI 13
4.7.4 PTT 13
4.7.5 CARRIER DETECT 13
4.7.6 RDSTAT 13
4.7.7 RESET 13
4.7.8 OPTION ID 13
4.7.9 MTR SPI Interface 13
4.7.10 SPARE UART 13
4.7.11 SPARE I2C 13

4.8 ST DEBUGGER INTERFACES (J1 & J11) 14

4.9 REAL-TIME STATUS INTERFACE (P5) 14

4.10 TX AUDIO TP (P6) 14

4.11 RX AUDIO TP (P7) 14
4.12 POWER (J7) 14
4.13 BATTERY JUMPER (J10) 15
5 BASIC SETUP 16
5.1 Factory Reset 16
5.2 Booting up with Factory Defaults 16
5.3 Initial Login after Factory Reset 16
5.4 Debug/logging interface 16
6 COMMAND INTERFACE 18
6.1 cls (cls) 18
6.2 help (hlp) 18
6.3 jitter (jit) 19
6.4 logout (exit) 19
6.5 mtrspi (msp) 19
6.6 netdelay (ndl) 19
6.7 rtc (rtc) 19
6.8 reboot (reboot) 20
6.9 rxtest (rxt) 20
6.10 settings (env) 20
6.11 set (set) 21
6.11.1 NETWORK SETTINGS 22
6.11.1.1 mystaticip (msi), mystaticsn (msn), mystaticgw (msg), mystaticdns (msd) 22
6.11.1.2 tftpaddr (tad) 22
6.11.1.3 myserialnum (msn) 22
6.11.1.4 Myipmode (mim) 22
6.11.1.5 guestpwd (gpw) 22
6.11.1.6 rootpwd (rpw) 23
6.11.1.7 s2nport (s2p) 23

6.11.1.8 s2nptime (s2t) 23
6.11.2 ALLSTARLINK SERVER SETTINGS 23
6.11.2.1 unauthrate (uar) 23
6.11.2.2 txdly (txd) 23
6.11.2.3 compression (cmp) 24
6.11.2.4 packettiming (ptm) 24
6.11.2.5 ipmode (ipm) 24
6.11.2.6 staticip (sip) 24
6.11.2.7 fqdn (qdn) 24
6.11.2.8 port (prt) 24
6.11.2.9 voterpwd (vpw) 24
6.11.2.10 hostpwd (hpw) 24
6.11.2.11 restart (rst) 25
6.11.3 GPS AND MISC SETTINGS 25
6.11.3.1 local (loc) 25
6.11.3.2 gpsdev (gdv) 26
6.11.3.3 gpsbaud (gbd) 27
6.11.3.4 gpsedge (ged) 27
6.11.3.5 gpsoffset (gof) 27
6.11.3.6 rxcal (rxc) 27
6.11.3.7 rxled (rxl) 27
6.11.3.8 rxmode (rxm) 27
6.11.3.9 rssimode (rsm) 28
6.11.3.10 logging (log) 28
6.11.3.11 probe (prb) 29
6.11.3.12 flags (flg) 30
6.11.3.13 myname 30
6.11.3.14 errlog 30
6.11.3.15 credits 30
6.11.3.16 all 31

6.12 show (show) 31

6.13 status (sta) 31

6.14 txtone (txt) 32

6.15 update (upd) 33

6.16 utest (utest) 34

7 APPENDIX A – FACTORY DEFAULT SETTINGS 35

8 APPENDIX B – SER2NET SETUP 36

9 APPENDIX C – TFTPUPDATE SERVER SETUP 39

9.1 DEBIAN tftp-hpa 39
9.2 WINDOWS OpenTFTPServer 39
9.3 DOWNLOAD FILE SETUP (COMMON) 40
9.4 REASON FOR A FIXED DOWNLOAD PORT 41
10 APPENDIX D – AUDIO LEVEL CALIBRATION 42
11 APPENDIX E - RSSI AND SQUELCH CALIBRATION 45
12 APPENDIX F - SQUELCH CALIBRATION 49
13 RELEASE NOTES 51

1 INTRODUCTION

This document describes the interfaces, functions, setup, and operation of the Voter2-2K. It is the companion document to the Voter2-2K Hardware Guide which focuses on the hardware design itself.

The Voter2 is designed to fully interoperate in a mixed environment with original Voters and an AllstarLink 1.x or 3.x server. Intended as a fallback only, it also includes basic stand-alone "local mode" repeater functions as well. While fully interoperable with the original Voter design, it is a completely new ground-up design effort. The reasoning for this is noted in the hardware document. The user interface and many functions may operate differently, but it is 100% faithful to the original Voter protocol (well, outside bugs of course which will be addressed over time).

This guide assumes the reader is already familiar with how the Voter ecosystem works and this guide is how to operate a Voter2 in that system. I hope over time this document (or another companion document) will describe the ecosystem as well. Until then, the two documents by Jim Dixon (WB5NIL, SK) "Voter System" and "Voter Protocol" are (and will remain) the reference documents.

Moore's Law is a wonderful thing. Over the 10 years the Voter has been out, silicon capabilities have increased considerably for a given budget. The Voter2 has 25x the CPU horsepower (50x if you count the bus width doubling), 64x the RAM, and 32x the flash memory of the Voter1 for roughly the same cost as Voter1 parts cost today. The Voter2 is designed with headroom, less than 10% of those noted resource being used with the initial feature set. The software architecture is based on an RTOS, so adding features and software maintenance should be easier as well. The feature set is expected to grow.

2 RELEASE STATE

This document revision is in sync with the similarly numbered software release version.

Several known less-critical feature omissions remain, these include:

- Locally implemented pre-emphasis on Tx audio (might be important)
- CTCSS detection (which may not be needed for the Voter2-2K specifically)
- AllstarLink server failover support
- Fully compliant TFTP support and more error checking on firmware downloads

Importantly, this is all-new hardware and software compared to the Voter/RTCM. It is absolutely not as stable and vetted as the Voter/RTCM that has millions of hours of use time. While most features are implemented, I am sure there are missing use cases still and it is not practical to test all combinations of all features, so there will be boundary cases that will crop up. These issues will be addressed as they are identified.

3 OTHER DOCUMENTS

Here are the full list of documents for the Voter 2. All are on the Github site, but in different directories depending on their association

- **Voter2 Users Guide:** (this document) Primarily centered around the setup and operation of the Voter2. Heavily tied to software releases. This document is in the software release directories and is in sync with the software release in that directory. In theory, this guide will be applicable to all Voter2s (Voter2-2K, Voter2-SA, possibly others)
- **Voter2 Hardware Reference Manual:** Primarily centered around the board hardware including board specifications, block diagrams, reasoning behind chip choices and describing the interfaces in detail. High level software architecture is covered as well. This document is in the hardware release directories and will be synchronized with the hardware revision level. The primary document will be for the Voter2-2K with the Voter2-SA being a "differences" document.
- **Voter2 Transition Guide:** This is a short document intended for folk already familiar with the Voter/RTCM and looking for a quick primer on how to get up and running with the Voter2. It highlights the similarities and differences from the Voter/RTCM. It is high level on purpose, not duplicating too much information in the other manuals. It is stored in "Other Documents" on Github.
- **Voter2 Quick Start Guide:** Another short document covering the core commands needed to bring up a Voter2 from factory defaults. It is stored in "Other Documents" on Github.

4 COMMUNICATION INTERFACES

This section describes each physical interface and then for interfaces like Ethernet and the backplane, each logical interface over that physical interface. Connections are listed from left to right looking at the front panel, then the back and finally internal connectors. Front panel space was scarce, forcing some tight constraints.

4.1 10MHz Reference

This was a backup 10MHz input that is not needed. It is intentionally depopulated and will be removed and/or replaced with something else in the future.

4.2 USB Console

This is a USB-C connector connected to an FTDI UART chip. "Regular" serial ports are alas extinct and FTDI drivers are native to most operating systems now, so hopefully this interface should be plug and play. Set the data rate to 115,200. The Voter2 should work with default settings for both PuTTY and Tera Term for all other settings. If you are using another terminal program (or changed defaults for the above two), the other major settings are:

- 8 data bits, 1 stop bit, No parity
- No implicit CRs or LFs
- No local echo ("auto" works fine too)
- No local line editing ("auto" works fine too)
- 0x08 or 0x7f for backspace (both supported)

The only line editing supported is backspace. I hope to enhance this in the future. The USB console and the Telnet console have effectively identical functions and are mutually exclusive. If one interface is active and a connection established on the other, the console will log out of the previous interface and then allow you to log into the interface just established. The unused interface goes dormant.

The USB console does have one "special power" the Telnet interface does not have, the ability to do a factory reset. See the section on logging in.

4.3 ETHERNET

This is a 10/100 full duplex Ethernet interface with status LEDs.

- Green: Connection speed, Off for 10Mbps, on for 100Mbps
- Yellow: Activity.

The MAC address uses the Locally Administered Unicast Address format which is essentially a random address. This is assigned at board manufacturing and programmed into the OTP area of flash, so permanently set.

The interface supports fixed IP addressing, DHCP and DNS. The Voter2 can be pinged.

Listed below are the currently supported Ethernet ports:

4.3.1 ALLSTARLINK Port

(UDP Client, random port) For comms with the AllstarLink server. For the Voter2 it is a random outgoing port number. For the host (the AllstarLink server) the server port is either 667 for ASL1 or 1667 for ASL3.

4.3.2 Telnet Console Port

(TCP Host, Port 23 – fixed) Telnet interface for the console. The Voter2 is set up to work with defaults for both PuTTY and Tera Term. If you are using another terminal program (or changed defaults for the above two), the other important settings are:

- No implicit CRs or LFs
- No local echo (negotiated using Telnet commands to disable)
- No local line editing (negotiated using Telnet commands to disable)
- 0x08 or 0x7f for backspace (both supported)

The only line editing supported is backspace. I hope to enhance this in the future. The Telnet and USB based console have effectively identical functions and are mutually exclusive. If one interface is active and a connection established on the other, the console will log out of the previous interface and then allow you to log into the interface just established. The unused interface goes dormant.

4.3.3 Logger Port

(TCP Host, Port 24 – fixed) The Voter2 has a tiered logging setup segmented by functional block (see the `set logging` command). The default interface on boot for this logging is the UART dedicated to the ST debugger interface. Logging into this port with a Telnet client will direct the logging data here instead of the UART, allowing remote logging. Disconnecting the Telnet client will return logging data to the UART.

Logging messages begin well before the IP stack is up and indeed even before the RTOS is active, so initial logging is always to the UART which is default on boot.

4.3.4 Ser2Net (RFC2217) Port

(TCP Host, Port 2000 – user changeable) RFC2217 is a protocol allowing a UART to be controlled remotely via a Telnet connection. Most repeaters including the MTR-2000 have a serial port for command/configuration. By using this protocol and a UART-to RFC2217 driver on the host PC, you can run the standard Motorola RSS software (yes only on 32-bit Windows) and be able to communicate with the MTR-2000 remotely.

The RFC2217 implementation on Voter2 does not support the complete RFC2217 feature set. It implements just enough to support the MTR-2000 software, namely baud rate and data format (data bits, parity, stop bits). No support is included for flow control or handshaking signals. If needed for future projects with different repeaters these features can be added.

Refer to Appendix B on how to set up the PC side software to work with the Voter2.

4.3.5 TFTP Port(s)

The Voter2 supports remote firmware updates via TFTP. The Voter2 is the TFTP client. See the `update` commands.

The Voter2 also supports settings backup and restore via TFTP. See the `settings backup` and `settings restore` commands.

Future features such as status and error logging may be enabled via TFTP.

4.4 MTR UART

This is the UART interface to the MTR-2000's RSS connector. An RJ45 connector right next to the Ethernet connector isn't great, but I did want to allow the use of an off-the-shelf cable between the Voter2 and MTR-2000. A standard but very short Ethernet cable will do the job. Please do yourself a favor and mark that cable as NOT ETHERNET.

The interface is standard RS-232 voltage levels but only connecting Tx, Rx, and ground. The MTR-2000 does not support any handshaking and neither does the Voter2. All communication parameters are controlled via the RFC2217 protocol.

4.5 LEDs

There are six status LEDs on the Voter2. Since this is a card in a card slot of the MTR-2000, they are not terribly easy to see and the silkscreens for the LEDs even harder to see. Here is a diagram that should help, viewed as you would see them from the front of the repeater.

ASL (yellow)	GPS (yellow)	Toward the back
RX (red/yellow/green)	TX (red)	
HB (green)	PWR (green)	Toward the front

The LEDs are as follows:

- PWR (green) – Power. On if there is 3.3V, not under any software control.
- HB (green) – Heartbeat. It has three normal modes:
    - Normal - Blinks once/second indicating normal operation.

  - Factory Defaults - During initial boot, if the NVM is blank or corrupt this LED will blink five times rapidly. See the note in the setup section regarding booting with factory defaults.
  - Bootloader – In Bootloader mode (final phase of doing a software update – the `update program` command), the LED will "reverse blink", mostly on, but blinking off to indicate system flash is being programmed. Once the programming is complete, the Voter2 will reboot and revert to normal blinking.
  - If the LED is stuck on or off, something bad has happened.
- RX – Receive Status. This is a three-color LED (plus off) indicating either the Rx level or COR/CTCSS status. See the command `set rxled` to select the mode. Stati are:

LED Color	Audio Level Mode	COR/CTCSS Mode
Off	Rx not active (based on `rxmode` settings)	Neither COR nor CTCSS active
Yellow	Rx active Audio < 1/3 full scale	COR asserted CTCSS not asserted
Green	Rx active Audio 1/3 to 2/3 scale	COR asserted CTCSS asserted
Red	Rx active Audio > 2/3 full scale	COR not asserted CTCSS asserted

  If in audio level mode, any color means Rx is active and during normal operation the LED should be bouncing between yellow and green with occasional flashes of red. In COR/CTCSS mode this shows the COR/CTCSS states. Whether it means Rx is active or not depends on how you set `rxmode`.
- TX (red) – Indicates the Voter2 is telling the MTR-2000 to transmit.
- ASL (yellow) – Indicates the AllstarLink connection state. Off for no connection, on for connected and authorized and flashing for connected, but not authorized (likely a bad Host Password). Note that a bad Voter password isn't obviously detectable, so won't cause the LED to flash.
- GPS (yellow) – Indicates the GPS link state. Off for no data, blinking for getting data but not yet 1PPS, and solid on for GPS data and valid 1PPS.

4.6 GPS(es)

The Voter2 supports two different GPS modules. Space constraints requires overlapping footprints, so only one or the other can be installed, there is no provision for installing both and selecting between them. The two GPS modules supported are:

4.6.1 Ublox M8n

This is a mezzanine board with a simple 5-pin interface. It contains the SMA connector and its own battery so completely self-contained. While the Ublox module is a good and reliable module, the mezzanine board is from suppliers that do not look like long term suppliers and offering complete boards for much less than purchasing just the Ublox module itself, so of "questionable origins". The ones we do have perform well, so this board supports them as long as we can get them.

Of note, if off for more than a day, it can take up to 30 minutes to get a 1PPS signal even though GPS position data is valid within a minute.

4.6.2 ST Teseo LIV4F

This solution is just the module. It is more expensive than the Ublox mezzanine board, but is a fully supported in-production module with guarantees and long term production commitments. While the Ublox module is opportunistic, this is the "10-year" solution. This requires an external SMA connector. While there is support for a battery for ephemeris data, its acquisition time including 1PPS is fast enough without it. The battery does not seem necessary.

4.7 MTR-2000 Backplane

The Voter2 can plug into either the Option 1 or Option 2 connector in the MTR-2000 card cage. Note that compared to the Voter, the wireline card is not required. The silkscreen on the bottom shows which pins have connections. If you are hand soldering, these are the only pins that need soldering. The supported signals on this connector are:

4.7.1 Rx Audio

Audio from the MTR 2000 to the Voter2. The Voter2 DC isolates it. Max signal amplitude is 2.2Vp-p. 2.2Vp-p represents full scale into the MCU ADCs.

4.7.2 Tx Audio

Audio from the Voter2 to the MTR-2000. The Voter2 DC isolates it. A full scale out of the DACs results in 1Vp-p on this line.

4.7.3 RSSI

Analog RSSI from the MTR-2000. The Voter2 digitizes this input and interprets it per the information on page 33 of the MTR-2000 field service guide. Note this is only used if analog RSSI is selected.

4.7.4 PTT

How the Voter2 tells the MTR-2000 to transmit. It is an open collector output of the Voter2.

4.7.5 CARRIER DETECT

Tells the Voter2 the MTR-2000 recognizes a valid carrier. This is only used if selected via the UI.

4.7.6 RDSTAT

Tells the Voter2 the MTR-2000 recognizes a valid CTCSS tone. This is only used if selected via the UI.

4.7.7 RESET

Allows the MTR-2000 to reset the Voter2. While provisions are made to connect this signal, it is not connected.

4.7.8 OPTION ID

This is an analog voltage to the MTR-2000 from the Voter2 for presence detect as well as identifying the classification of the board.

4.7.9 MTR SPI Interface

This allows the MTR-2000 to set or query 16 control bits. There is no known use for this feature (yet), but supported in hardware. The MTR-2000 is the SPI bus master, the Voter2 is the slave. There are separate chip selects for read and write, but they share the CLK, MISO, and MOSI lines. Additionally the OPT_IRQ signal from the Voter2 to the MTR-2000 can be asserted to force a read of the read bits.

4.7.10 SPARE UART

This is an extra UART sent to the backplane intended for an S-COM board that will plug into the MTR-2000 System connector. It is currently not used by the Voter2.

4.7.11 SPARE I2C

This is an extension of the internal I2C bus sent to the backplane intended for an S-COM board that will plug into the MTR-2000 System connector. It is not directly used by the Voter2. If, however, you have some extra P3T1755 temperature sensors hooked up to

these lines with unique addresses, they will be detected and temperatures displayed on request.

4.8 ST DEBUGGER INTERFACES (J1 & J11)

The Voter2 supports two debugger interfaces. The 14-pin interface is a full featured interface supporting regular debugging, instruction tracing, and the UART logging. The baud rate for the UART logging is 115,200 baud. This port requires a higher end debugger dongle for these features. For a simpler and MUCH cheaper debugging interface, the 6-pin interface matches the connector on STs Nucleo modules. This connector doesn't support instruction tracing or the UART logging interface, but fully suitable for regular debugging.

4.9 REAL-TIME STATUS INTERFACE (P5)

For real-time monitoring of critical software functions such as ISRs and DSP routines this connector provides three GPIO lines that can be wiggled by software for viewing on an oscilloscope. They are labeled AISR for Audio data processing, TISR for Timer interrupt processing and DISR specifically for measuring DSP routine execution time. These are just GPIOs, so can (and are) routinely changed to measure specific pieces of critical code execution time.

The fourth output is an analog output synchronized with the audio DSP chain. Using the `set probe` CLI command, you can probe the audio signal at any of the stages of the DSP chain to verify the signal quality/integrity. It is the DSP equivalent of probing the signal at the output of the various filters if they were implemented as circuitry. See the `set probe` command for details.

4.10 TX AUDIO TP (P6)

Analog audio from the Voter2 to the MTR-2000 after DC isolation (so centered around 0V) before the 600 ohm source impedance resistor. It has a matching ground pin for better signal measurement. This is the point where full scale should be 1Vp-p.

4.11 RX AUDIO TP (P7)

Analog audio from the MTR-2000 to the Voter2 after DC isolation (so centered around 1.65V). This is the point where full scale should be 2.2Vp-p.

4.12 POWER (J7)

A place to check all four voltages, +5 digital, +5 analog, +3.3V digital, and +3.3V analog. Both analog and digital grounds are present as well, but note that the common tie point between the two is very close to this connector.

4.13 BATTERY JUMPER (J10)

If the battery is not being used, this jumper supplies +3.3V digital to the 3.3V battery bus so the battery-operated circuitry has a valid voltage whenever the board is powered. Clearly do NOT install this jumper and a battery at the same time!

5 BASIC SETUP

5.1 Factory Reset

To restore the Voter2 NVM to factory defaults requires you log into the console over the local USB interface. When you get to the login prompt, enter a username of `factory` and a password of `reset`. This will purposely corrupt the NVM so that on the next reboot it will load factory defaults instead of NVM settings. A list of the factory default settings are in Appendix A.

5.2 Booting up with Factory Defaults

When the Voter2 boots up with a blank or corrupt NVM you will see the heartbeat LED blink rapidly five times before reverting to the normal once-per-second flash. When the Voter2 boots with factory defaults, it loads them into local memory only, IT DOES NOT UPDATE THE NVM! This is on purpose. This allows you to inspect the NVM if you wish first. As such, after logging in and making any changes, you must issue the `settings save` command before the next reboot or you will end up repeating the process.

5.3 Initial Login after Factory Reset

A voter2 powering up with factory defaults has the following important settings for getting started with the command line:

- USB UART: 115,200 baud
- Telnet: DHCP for IP address, port 23.

Figuring out the DHCP-assigned address is messy, so the easier way to get started is to use the USB UART. Odds are very good your OS has FTDI drivers already, if not you will need to get the drivers for the FTDI FT230XS. There should be no reason to change any other defaults for either PuTTY or Tera Term.

There are two hard-coded login accounts for the Voter2, `root` and `guest`. The factory default passwords for them are `guru` and `password` respectively. The full list of factory defaults are in Appendix A. The usernames and associated passwords are case sensitive. The main difference between the two accounts is that only root can view/change any of the passwords (root, guest, and AllstarLink/voter passwords. Some other functions such as I2C memory dump and possibly Ser2Net settings may be limited to root access. Saving settings to NVM Is not allowed in guest mode, making it harder for a guest to permanently break the Voter2. Anything the guest does can be undone by a reboot.

5.4 Debug/logging interface

A big difference from the Voter is the Voter2 uses a separate interface for debugging and logging messages than the console. This defaults to the UART on the 14-pin ST Debug

connector, but you can also access it via Telnet on port 24. See the sections above regarding these two interfaces. The command to control logging is `set logging`.

6 COMMAND INTERFACE

Unlike the Voter, the Voter2 uses text based commands with a rudimentary help system to provide reference. This is more verbose than the numeric interface, but more flexible. In general, CLI commands and text parameters are case insensitive. String parameters such as usernames, passwords, filenames, and CW IDs are case sensitive. The first useful command is `help` (shows all possible commands). Help followed by a command shows the syntax for that command. The most likely initial commands are going to be for settings, so `help set` will show you all the settings that be changed. The `show` command will show the value of any of those settings. While a bit clunky, there is also a `show all` command that lists all settings with one command, get ready to scroll back.

If you make any changes to settings (or came up with factory defaults), these are only stored in local RAM, not NVM. There will be an asterisk before the > of your command prompt. That asterisk is a reminder that you have settings in local memory that have not been saved to NVM and will be lost if you reboot. This allows a very limited "undo" by simply rebooting the Voter2.

The command line interface supports last-command recall for up to ten commands. You request these by using the up (previous) and down (newer) arrow keys on your terminal. This assumes your terminal sends ANSI "arrow" codes for those keys. Putty and Tera Term do, so this shouldn't be a stretch. Currently the only editing supported is backspace, there isn't support for inserting/deleting characters inside a line.

Next is the reference list for the current commands on the Voter2. Commands are in alphabetical order, but for settings they are grouped and listed as they are in the help listing. Of note, while this manual has been recently updated to call the server as an AllstarLink server, the source code and Voter2 CLI still refer to it as the DIAL server.

Most commands and most "list parameters" have a shortcut available to save typing. The shortcut is noted here and in the CLI help in parenthesis.

6.1 cls (cls)

Send the ANSI/VT100 FF (Formfeed, 0x0c) byte to the console. If you have almost any kind of terminal emulation this should clear the screen.

6.2 help (hlp)

`help` by itself lists all commands.
`help` followed by a command lists the syntax for that command.
`help set` or `help show` lists all the set and show parameters.
`help set` or `help show` followed by a setting will give the syntax for that setting.

6.3 jitter (jit)

This command measures inter-packet jitter, handy for determining how predictable your network is. A histogram is presented over a range of packets. Ideally, audio packets come in (when they do come in) every 20 or 40ms depending on the audio compression mode. This is true for both GPS-timed and General Purpose packets, so this command works with both modes. This command allows a jitter measurer to `start`, `stop`, `reset` and `display` a histogram of that inter-packet time. To prevent overflows, if any bin gets to a count of 50K, the histogram tracker stops automatically. There is a companion command called `netdelay` that is just for GPS-timed packets and measures absolute times.

6.4 logout (exit)

Logs out of the console session. You are returned to the login prompt. The connection (UART or Telnet) is not closed. If done, just close your terminal program afterwards.

6.5 mtrspi (msp)

This is to test the MTR-2000 SPI interface. There is no known use for this interface on the Voter2, it is for the S-COM project. This interface can affect the functions of the MTR-2000, so don't mess with it!

6.6 netdelay (ndl)

This command measures the time difference between the timestamp of a just-delivered GPS-timed audio packet and the current GPS time and presents that as a histogram over a range of packets. As a packet gets timestamped at its creation, the difference between the packets timestamp at arrival and the current GPS time represents the round trip time for that packet via the AllstarLink server. As this uses timstamps and GPS, this only works for GPS-timed packets. This is handy for determining what your `dial txdly` setting should be. This command allows the delay measurer to `start`, `stop`, `reset` and `display` a histogram of that round trip delay. To prevent overflows, if any bin gets to a count of 50K, the histogram tracker stops automatically. There is a companion command called `jitter` that measures just jitter, not absolute delays and works for both General Purpose packets and GPS-timed packets, it just doesn't give absolute times.

6.7 rtc (rtc)

This feature and commands are only useful if a battery has been installed. For the Voter2 where a super-precise time is available via GPS and/or an AllstarLink server, it is likely not useful. If the Voter2 is operating as a stand-alone device in local mode only, then this can be the time reference. While settable and viewable in this command line interface, it is currently not used anywhere else in Voter2 code. If there is no battery and the sync

command hasn't been issued, the RTC value is essentially the "time since power-up". Three RTC commands are available:

`sync`: If you do have valid GPS time, this command will program the RTC with the GPS time. This will be UTC time.

`set yy mm dd hh mm ss`: Manually set the RTC time.

`show`: Show the current RTC time.

6.8 reboot (reboot)

Reboots the Voter2.

6.9 rxtest (rxt)

This parameter enables test patterns and tones on the receive side of the Voter2 (patterns that get fed to the AllstarLink server). As these tones are injected as if they are coming from the ADC, they are subject to normal Rx timeout restrictions. Currently four test patterns exist. They are:

- `off`: Turn any test pattern mode off and resume regular operation.
- `tp20`: Timing pulse every 20ms (1 packet)
- `tp80`: Timing pulse every 80ms (4 packets)
- `tp220`: Timing pulse every 220ms (11 packets)
- `dev3k`: 1kHz tone emulating an incoming signal with 3kHz deviation

The timing pulse patterns are for verifying and calibrating simulcast functions. The reason for several is to guard against a "false match" if you have a delay that exactly matches the pulse period. If the 80ms and 220ms pulses both line up, you would have to be off by a multiple of 880ms to get a false match.

The 1kHz tone is to emulate the expected signal of a well calibrated 1kHz tone at 3kHz modulation going through a repeater into the Voter2. It is a convenient way to check/calibrate the rest of the path through the AllstarLink server without needing a service monitor setup.

6.10 settings (env)

This command is to manage saving and restoring settings as a whole. If you have modified any individual setting or came up in factory default mode, then your settings in local memory are not in sync with what is in NVM. If that is the case, you will see an asterisk before your ">" command prompt. If you reboot or enter the `settings reload` command those local settings will revert to the ones in NVM. Note though that settings are not reloaded if just logging out and back in again.

Additionally, settings can be sent to a TFTP server or restored from a TFTP server. Note that these settings are loaded from or stored in local memory only, not NVM. If you want settings loaded via TFTP to be in NVM as well, you will need to follow the `settings restore` command with `settings save`. The settings are saved in an ASCII text file as a series of set commands just as they would be typed into the CLI. This allows you to edit as you see fit to clone a Voter2 configuration either entirely or modify select parameters only.

The settings commands are:
`save`: Save current settings in local memory to NVM. The total number of bytes written is noted, get worried if it is getting close to 8192. This command is only available in root mode. This allows a guest to try settings, but can't make any permanent changes.

`reload`: Revert current settings (if changed) back to what is in NVM. The total number of bytes read is noted. This command is available in root and guest modes.

`dump`: This is a debugging command to dump the contents of a range of NVM. If no additional parameters are provided, byte addresses 0-255 will be shown. Valid start and end addresses for the 24LC64 are 0-8191. This command is only available in root mode.

`backup`: Back up the settings to the TFTP server (same IP as for software updates). The settings will be saved in a file named `Voter2-2K_SN####.txt` where #### is the serial number of the Voter2. Optionally, you can specify your own filename and it will be used instead. Be aware of fun OS-specific things like case sensitivity! This command is only available in root mode. A text file of CLI commands was specifically chosen so you can edit the file on your computer if needed.

`restore`: Restore settings from the TFTP server (same IP as for software updates). The settings will be loaded from a file named `Voter2-2K_SN####.txt` where #### is the serial number of the Voter2. Optionally, you can specify your own filename and it will be used instead. Be aware of fun OS-specific things like case sensitivity! As this command is just feeding the file into the command line interpreter, this could also be a general batch file containing any commands you care to add on your computer, not just `set` commands. This command is available in root and guest modes. Note also that commands are interpreted as they read so if there is any kind of file error, settings up to that error will have been interpreted. There is no "undo" even for an erroneous restore.

6.11 set (set)

This command and its companion `show` command allow you to set and show the operating parameters of the Voter2. Settings changed with the `set` command are only

changed in local volatile memory. They are not automatically saved in NVM. If you want the changes to be permanent (across reboots), the settings should be saved to NVM using the `settings save` command. If a setting requires a reboot to take effect, this will be noted in command feedback.

The factory default settings are listed in Appendix A.

Commands listed below are by functional grouping instead of alphabetical and are in the same order as the `help set` string.

The help function works for each of the settings. To get the syntax for a specific setting, enter `help set [setting]` to get that settings syntax.

6.11.1 NETWORK SETTINGS

6.11.1.1 mystaticip (msi), mystaticsn (msn), mystaticgw (msg), mystaticdns (msd)

If `myipmode` is set to `static` these will be the Voter2's IP, subnet, Gateway, and DNS server addresses respectively. The parameter is the standard xxx.xxx.xxx.xxx format where xxx is 0-255. Ignored if `myipmode` is set to `dhcp`.

6.11.1.2 tftpaddr (tad)

The IP address of the TFTP server for software downloads and NVM backup/restores.

6.11.1.3 myserialnum (msn)

The Voter2's serial number. This is in OTP, so show only. If the Voter2's OTP is not yet programmed, then this is the command to program all OTP values, the serial number, hardware revision, and MAC address. In this case, there are three parameters, the serial number (32-bits, so huge), the hardware major revision 0-255, and the hardware minor revision (0-99). The MAC address is not a parameter, it is randomly generated, locally administered, unicast MAC address (that's an official thing!). As these are OTP values, the command to set them can only be executed once and will normally be executed as part of board manufacturing. After that, this command is show only.

6.11.1.4 Myipmode (mim)

- Voter2 IP address acquisition mode. It can be `static` or `dhcp`.

6.11.1.5 guestpwd (gpw)

This is the password for the Voter2 guest login. It can be up to 16 characters and is case sensitive.

6.11.1.6 rootpwd (rpw)

This is the password for the Voter2 root login. It can be up to 16 characters and is case sensitive.

6.11.1.7 s2nport (s2p)

Ser2Net is the protocol for remote UART connections managed over Ethernet using the RFQ2217 standard. This is the listening port for that service. The default is 2000, but this command can change it. It is any valid port number, 0-65535.

6.11.1.8 s2nptime (s2t)

This is the polling time for serial data from the MTR-2000. The default of 100ms works with the Motorola software. If you are using different software that may be more sensitive to timing issues, this value can be decreased at the expense of more CPU time. If you tweak it, do try for the longest possible time that does not generate errors.

6.11.2 ALLSTARLINK SERVER SETTINGS

Note that the AllstarLink server settings are only available if AllstarLink mode support is enabled! Clearly the case for the Voter2, but not so if being used for non-Voter applications.

6.11.2.1 unauthrate (uar)

This is the rate (in ms) to send authorization packets to the server when in the unauthorized state. The default is 1000ms. It would be strange to change this, but it can be if needed.

6.11.2.2 txdly (txd)

This is the amount of transmit audio data to buffer up in ms before asserting PTT and transmitting audio. This value is highly dependent on your network setup and in general you are trying to make it as small as possible without buffer underruns.

With simulcast support this delay is not only responsible for handling network jitter but also any tweaking needed to better line up simulcasting transmitters. While the unit of measure remains milliseconds in the UI (easier to grasp), internally it is stored in ns. For the UI it is now a “floating point” number with up to six digits after the decimal point (but no more), then converted to ns. The timer generating the audio samples runs at 40MHz, so deltas of less than 25ns have no effect. The GPS 1PPS references signals are generally within ±100ns of each other, so deltas of less than 200ns will not be repeatable.

Acceptable values for this parameter are 60-800ms. You do want this value as low as practical as this time adds directly to system delay.

6.11.2.3 compression (cmp)

The Voter protocol supports two Audio compression standards, uLaw and IMA ADPCM (Intel/DVI block format). The AllstarLink server tells the Voter2 which format to use so this command doesn't do anything! (needs work).

6.11.2.4 packettiming (ptm)

Tells the Voter2 what packet timing mode to use. The official options are `general` for General Purpose mode also known as mix mode or non-GPS mode, or `gps` for GPS-timed mode. This must be `gps` for voting operations. A third mode, `fakegps`, is for test purposes where the Voter2 gets the time from the AllstarLink server and then fakes gps packet timing using that time. This latter mode should not be used outside bring-up testing.

6.11.2.5 ipmode (ipm)

The AllstarLink server IP acquisition mode. It can be `static` or `dns`. If `static`, then the AllstarLink `port` parameter is used. If `dns`, then the address is determined using dns and the `fqdn` parameter.

6.11.2.6 staticip (sip)

The IP address of the AllstarLink server if `ipmode` is set to `static`. Ignored if `ipmode` is set to `dns`.

6.11.2.7 fqdn (qdn)

If the IP acquisition mode for the AllstarLink server is DNS, this is the FQDN (path) for the DNS server to look up. It can be up to 40 characters.

6.11.2.8 port (prt)

The UDP port number for AllstarLink communications. This should be 667 for AllstarLink servers up until ASL3, then 1667 for ASL3 (and probably later).

6.11.2.9 voterpwd (vpw)

For Voter/AllstarLink authentication, this is the voter password. It is the unique per client password called out in the line for each client and in the Voter UI as parameter 5. It can be up to 16 characters.

6.11.2.10 hostpwd (hpw)

For Voter/AllstarLink authentication, this is the host password. In the ASL voter.conf file, this is the global password identified by "`password =`" and in the Voter UI by parameter number 6. It can be up to 16 characters.

6.11.2.11 restart (rst)

This will reset the AllstarLink connection state and stop all AllstarLink comms for about 5 seconds for things to reset, then allow AllstarLink comms to restart. No other IP ports are affected.

6.11.2.11.1 ctcss (pl)

Controls enabling the CTCSS tone for ASL modes. See `ctcssfreq` and `ctcsslvl` for CTCSS tone settings. Parameter can be `none`, `gp`, `gps`, or `both` for neither mode, General Purpose, GPS-times, or both modes respectively.

6.11.3 GPS AND MISC SETTINGS

6.11.3.1 local (loc)

If communications with the AllstarLink server are down, the Voter2 reverts to Local mode for stand-alone operation. This is a minimal repeater controller operation. As soon as comms with the AllstarLink server are restored, the Voter2 goes back to Voter mode. The settings below apply to implementing the stand-alone local mode.

6.11.3.1.1 mode (mod)

The operational mode while in Local Mode. These are the same modes as the Voter and are:

- `none:` The Voter2 does not transmit anything, even if receiving a local signal.
- `simplex:` The Voter2 behaves as a simplex repeater. Without an AllstarLink connection this results in the "OFFLINE" message on disconnect and CWIDs being sent as needed in response to received audio, but no audio.
- `trigger:` Same as simplex, but only sends the "OFFLINE" and CWID once, then silent until an AllstarLink connection is established.
- `repeater:` The Voter2 becomes a simple stand-alone full duplex repeater controller.

6.11.3.1.2 cwspd (cws)

The speed at which to send CW identifiers, the station ID and proceed ID. Valid range is 5-50. In wpm.

6.11.3.1.3 deemph (dmp)

Enable/disable De-Emphasis in local mode. Setting can be `on` or `off`.

6.11.3.1.4 precw (prc)

The delay between the end of any repeated audio and the next ID which could be the proceed ID or station ID. In ms.

6.11.3.1.5 postcw (poc)

The delay between the proceed ID and station ID if both are sent. In ms,

6.11.3.1.6 idtime (idl)

The time between station ID transmissions if anything was repeated. FCC suggests 10 minutes, so unless you are doing something weird the default to 600 seconds is normal.

6.11.3.1.7 hangtime (htm)

The delay from whatever was last to be transmitted (audio, proceed tones, ID, etc) and releasing PTT. In ms.

6.11.3.1.8 txdly (txd)

This is buffer delay time in ms when in local mode. Buffer delay time can be different when in local mode vs in AllstarLink mode. While this parameter is in ms, the Voter2 "rounds" this up to the nearest packet length of 20ms. A length of 1-20ms will result in a 20ms delay, a value of 21-40ms a 40ms delay, etc. Also, unlike the `txdly` parameter for AllstarLink mode, this parameter is purely in integer ms. This parameter can generally be much shorter than AllstarLink mode as all data is locally processed. Valid values are 60-800ms.

6.11.3.1.9 id (id)

The local callsign ID string. This can be the characters A-Z, 0-9, - or /. The max length is 10 characters. It should be all upper case.

6.11.3.1.10 proc (prc)

The local proceed ID string (sent after every transmission) in local mode. This can be the characters A-Z, 0-9, - or /. The max length is 10 characters and should be upper case.

6.11.3.1.11 ctcss (pl)

Controls enabling the CTCSS tone for local mode. See `ctcssfreq` and `ctcsslvl` for CTCSS tone settings. Parameter can be on or off.

6.11.3.2 gpsdev (gdv)

This is a convenience shortcut command to set up the GPS receiver parameters by device, saving you the effort of looking them up. It assumes you haven't changed the defaults of the GPS! GPS modules currently supported are `m8n` and `liv4f` for the Ublox M8N and ST Teseo Liv4f respectively. The settings changed are `gpsbaud` and `gpsedge`.

6.11.3.3 gpsbaud (gbd)

Sets the baud rate of the GPS receiver. Parameter can be between 2400 and 115200.

6.11.3.4 gpsedge (ged)

Selects the active edge of the GPS interfaces 1PPS signal, the edge representing the time in the NMEA message. Parameter can be `rising` or `falling`.

6.11.3.5 gpsoffset (gof)

This parameter allows you to apply an offset to the GPS-based time for timestamps on sent and received packets. The reason for this is to interoperate with some older GPS receivers that no longer report the correct time. The parameter is integer seconds and is a 32-bit signed integer, so -2147483648 to 2147483647. On purpose, this offset is applied without any range or overflow checking as when dealing with Unix time near the epoch, weird things start to happen! Use it carefully if at all. In addition to a numeric offset the Voter2 also supports known offsets (just one so far)

- `thunderbolt`: Offset of -TBD seconds for the Trimble Thunderbolt GPS.

6.11.3.6 rxcal (rxc)

This is for calibrating the receive analog RSSI range. See Appendix E for details on how to run this calibration.

6.11.3.7 rxled (rxl)

The Rx LED has two user selectable modes. See section 4.5 (LEDs) for the full description of these modes. Parameters are:

- `level`: Show Rx state and audio level.
- `status`: Show COR and CTCSS states

6.11.3.8 rxmode (rxm)

This selects what signals and conditions constitute a valid received signal (that is, means the audio gets sent to the AllstarLink server or rebroadcast if in local mode. Parameters are:

- `off`: Never receive. For a TX-only site.
- `cor`: Rely on the MTR-2000 COR signal only (which is carrier detect)
- `ctcss`: Rely on the MTR-2000 RDSTAT signal only (which is valid CTCSS).
- `corctcss`: Rely on both COR and CTCSS being valid.
- `on`: Transmit always (!!!). For testing purposes. Note that this mode still gets overridden by the timeout timer in local mode.

When I get the out-of-band RSSI function working this will get a bit more complicated.

6.11.3.8.1 ctcssfreq (ctf)

The desired transmit CTCSS frequency if enabled. See `set local ctcss` and `set dial ctcss` for enabling CTCSS. The standard CTCSS frequencies are supported:
`67.0 71.9 74.4 77.0 79.7 82.5 85.4 88.5 91.5 94.8 97.4 100.0 103.5 107.2 110.9 114.8 118.8 123.0 127.3 131.8 136.5 141.3 146.2 151.4 156.7 162.2 167.9 173.8 179.9 186.2 192.8 203.5 210.7 218.1 225.7 233.6 241.8` and `250.3`.

6.11.3.8.2 ctcsslvl (ctl)

Volume level of the local CTCSS tone. Value is 0 (none) and 1-10 are percent full scale. The CTCSS tone is added to the audio level. See `set local ctcss` and `set dial ctcss` for enabling CTCSS.

6.11.3.9 rssimode (rsm)

This setting selects what source to use for determining the RSSI value to send to the AllstarLink sever. There are four possible modes:
- `analog`: Use the RSSI value from the MTR-2000 itself (it is an analog signal)
- `hpf`: Determine RSSI by looking at energy above 2.4kHz. With a good signal there shouldn't be any energy there, so if there is, it is probably noise. This is the how the Voter works.
- `bew1`: Use the DSP/BEW algorithm with attenuated audio.
- `bew2`: Use the DSP/BEW algorithm with clipped audio.

The DSP/BEW algorithms are pretty involved. Rather than try to summarize/duplicate a description, best to see good online docs for a fuller understanding. The short version is that if there is a sudden loud noise, it can be interpreted as loss of signal. These BEW modes note that noise and hold the pre-noise RSSI until the noise passes.

6.11.3.10 logging (log)

The Voter2 uses a separate interface for posting logging information rather than sharing it with the console. See the sections on the Ethernet logger port and ST Debug ports for descriptions of where logging information is sent.

The Voter2 has a more granular approach to logging than the Voter1. There are only four logging levels, but logging can be set uniquely for each functional block, allowing more or less detail by block, not for everything. In general the levels mean:
- `none`: No messages at all from this block.
- `major`: Only major (startup and important error) messages.
- `support`: More granular information about good data too.
- `io`: Lowest level of detailed information (lots of data!)

The defaults for all blocks are major except for boot which is support. Logging levels are not saved across reboots.

The logger support tries to gracefully manage large impulses of data. It has a ring buffer currently set to 1024 bytes and if data comes in faster than the UART can empty it and the buffer fills, logging messages get dropped. While this is possible too with using the Ethernet port for logging, that should be tough to overflow.

The current functional blocks that have logging support are:

- `boot`: The boot process
- `mainapp`: This shows usage statistics mostly, once a second in support mode
- `ethapp`: Voter2 Ethernet support code
- `ethlwip`: LWIP (ethernet stack) code
- `ksz8081`: Ethernet PHY code
- `console`: Internal console parsing and character handling.
- `i2c`: I^2C comms debugging
- `dial`: AllstarLink server interface
- `gps`: GPS data and 1PPS interface
- `logtel`: Logger (logger, log thyself!)
- `local`: Local mode
- `ser2net`: Ser2Net interface
- `spinor`: SPI NOR interface
- `spimtr`: MTR-2000 SPI interface (not used by Voter2)
- `vcxo`: VCXO control loop (not in production yet)

6.11.3.11 probe (prb)

While DSP is great for flexibility, it's really hard to probe the intermediate signals between the various DSP blocks in real time since they are just function calls inside a program. This command helps with that. At each stage in the Rx and Tx DSP chains there is a monitor function that if selected will show what the signal looks like at that stage and drive it to a second DAC so you can look at the signal at that stage on your oscilloscope. Pretty cool, eh? This signal is available on pin 4 of header P5. Note that due to shared resources, the analog signal is only available if also transmitting. This is the command that selects where to probe. Probe points are:

- `none`: Probing disabled. This is the power-up default.
- `rxin`: Rx signal as digitized by the A/D (no DSPing yet)
- `rxdem`: Rx signal after de-emphasis
- `rxvoice`: Rx signal after the voice passband filter (300-2400Hz)
- `txsrc`: Tx signal as it came from the AllstarLink server post decompression. Or the locally repeated audio if in local mode (identical to `rxvoice` in this case)
- `txpl`: Tx signal after CTCSS adder (if enabled)

- `txtone`: Tx signal after test tone insertion (if enabled)
- `txout`: Tx signal as it's going to the DACs. Well, not really, in making the code efficient, I don't have a dedicated buffer for just the DAC samples so this is the same as `txout` (the last buffer in the DSP chain).
- `rsssig`: The Rx signal after the high pass filter for RSSI determination
- `rsslvl`: The derived RSSI value from the RSSI energy detect. The "0-255" level is translated to 0-3.3V on the DAC for the entire packet.

6.11.3.12 flags (flg)

This sets a value for internal debugging flags. Unless you are a code maintainer or in close contact with one, best not to tinker with this command! :) The first parameter is the flag number and second parameter the flag value. While easily changed, there are currently 8 flags (0-7) and each flag can have a value of 0-255. The default values on power-up and reboot is zero for all flags and values are not preserved on reboot.

Their use (if at all) is purely for transitory internal debugging and likely will change with each code release. Debugging code may also change the value of a flag to something different than was set.

6.11.3.13 myname

This sets a name to be echoed at the command prompt, making it easier to know which Voter2 you are dealing with in a network with many of them. Good idea Dave!

6.11.3.14 errlog

This command manages Voter2 error logs, mostly for long term Ethernet traffic error tracking. It is for low level debugging, not for general users, so somewhat cryptic. It is an array of error counters for various internal functions. If any errors occur, a counter for that error gets incremented. Showing this parameter lists any non-zero counter values with just an error counter ID and count. "Setting" this parameter (no data) resets all the counters to zero. Note that these counter values are preserved across resets, so likely have garbage values on initial power-up. To be useful the counter values must first be manually reset to zero. Additionally, if the Voter2 has a battery installed, then these count values will even be preserved across power cycles! If you are curious, the index of IDs are in `ErrLog.h`. Locate the enum entry for a counter that is incrementing and then search for that enum in the source directory.

6.11.3.15 credits

Just what it says, the Voter2 sponsors and development team.

6.11.3.16 all

A quick and easy way to print out all the settings to capture and save. (show only)

6.12 show (show)

The companion to the `set` command. See it for all the parameters.

6.13 status (sta)

This command is to show the status of different systems and interfaces of the Voter2. Nothing is settable. The items to show are:

- `10mhz`: Shows the frequency of the local 10MHz clock using the GPS1PPS as the reference (which it is!). For Voter2s with "regular" crystals, it shows the measured frequency each second along with the error in ppm (parts per million). For Voter2s with VCXOs the frequency is an average of the last 25 seconds in order to get the necessary accuracy and the error is shown in ppb (parts per billion). For Voter2s with a VCXO, the command `show logging vcxo` will provide greater visibility of workings of the VCXO control loop.
- `14v`: Shows the status of the 14 Supply. It is just a comparator at 11.9V, so not very useful.
- `dial`: Shows the status of the connection with the AllstarLink server including all of the status byte settings.
- `ethernet`: Shows the Ethernet MAC address and link status. If connected, it also shows the link speed, duplex, and resolved IP address.
- `gps`: Shows the status of GPS receiver including 1PPS. Note that GPS data is continuously arriving so sometimes data can get out of sync. This is particularly true for the number of satellites and satellite signal strengths.
- `rxfft`: Shows the results of an FFT of the received signal in waterfall format. This is for development and debugging of the DSP/BEW algorithm and not really useful outside that purpose. Letters A-P represent the frequency bins from 0, 250Hz, 500Hz…3.75kHz. Their position is higher amplitude to the left. If there is an overlap, the highest frequency "wins". The scale is arbitrary (and not linear). To the right is a flag that shows a B if the BEW triggered and an energy number representing certain noise levels.
- `level`: Shows a live portrayal of the incoming audio level. The left "waterfall" is absolute audio level. It is scaled to a 0-255 number with 255 being a full scale of the DAC input. The bar "tick" marks are in units of 50. For normal operation, this should never exceed about 150. The right waterfall is the audio level zoomed in on spot where a 1kHz tone with 3kHz deviation should be. See Appendix D for details. The display keeps on updating every 200ms until a key is received via the console.

- `power`: Shows the state of the MTR-2000s AC_FAIL signal. If the Voter is running and AC_FAIL is active, that means the repeater and Voter2 are running from battery power.
- `rssi`: Shows both the analog and high-pass computed RSSI values. The bar "tick" marks are in units of 50. The display keeps on updating every 200ms until a key is received via the console. To the far right are the status of the repeaters COR and RDSTAT signals. They are periods if not asserted and C or R if asserted. RDSTAT – if programmed is the status of the repeaters CTCSS detector.
- `rtos`: Shows CPU utilization of all RTOS tasks. This is currently the direct text output of an internal RTOS function, so not alterable. The first column is 1ms sample ticks and the second percentage. It shows utilization since boot, is not resettable, and the documentation says it can overflow within hours (for me it seemed good for 4+ hours) but still not general purpose. This command will likely evolve. Enabling mainapp logging reports (amongst other things) the percent CPU utilization every second for just that past second, so probably what you are looking for. Enter `set logging mainapp io` and look at wherever your logs go to see this information.
- `sync`: Checks the synchronization between the Voter2 and the AllstarLink server's time. This is not an accurate test and intended for a rough check only. Healthy numbers range from 500 to 1500us on a local network.
- `tempr`: Shows the internal temperature of the MCU as well as any external I2C temperature sensors. There is one on the board away from the MCU and others may be added off-board.
- `uptime`: Shows the time since last boot. Up to 944 days, the limit of a counter I'm using. If the Voter2 is up for that long, happy days!
- `version`: A consolidated list of major hardware and software revision numbers.

6.14 txtone (txt)

This controls the transmit side test tone generator. A sinewave with the specified parameters will be generated. When enabled, PTT is asserted and any Rx or AllstarLink audio overridden. Rx timeout is ignored too, so when enabled, it stays so until disabled here. Parameters are:

- `Off`: Turns off the tone generator if turned on.
- `Level`: The amplitude of the tone. If the tone generator is off, setting a level turns it on—even if set to 0 (silence). The value is in percent full scale.
- `Freq`: Frequency of the tone in Hz. This can be 1-4000.

If you enter ttone with no parameters, the current tone settings will be displayed.

6.15 update (upd)

This is the command to remotely update the Voter2 firmware. Refer to Appendix C for how to set up a TFTP server to support firmware downloads. The Voter2 currently needs some special tweaks to the TFTP server setup to work, so please read and implement the Appendix fully before executing any of the Voter2 commands below. The Voter2 uses a staging flash to hold the new image before programming the executable flash, that way it is not relying on anything other than stable power for the executable flash update process. Updating firmware is therefore a multi-step process: First is to check for the latest version on the server, second to download a new version into the local update flash, then finally to update the executable flash from the download flash. Update commands are as follows: (in the order you would run them)

- `check`: Check the update server to see if there is a newer version. The current version running on the Voter2 and the version on the server will be listed so you can compare them.
- `Download [filename]`: Download the version noted in check command into local download flash. The preferred (and safer) way to do downloads is to use the `check` command first as it supplies the vetted filename to download. You can optionally specify your own filename here to override what was recorded with the `update check` command. Be careful with this override, you can brick the Voter2 with wrong download image. This does not update the Voter2, it just stages the image into a staging flash to ensure we have a complete and verified image in local memory before doing the update.
- `Verify [filename]`: Verify the code in the download flash matches what is available on the server. As with `download`, the check command should be performed before this command or you can optionally specify your own filename to override what was recorded with the `update check` command. In theory this step isn't needed as the download command is also checking, but this is still handy to run. If you ever get a verify error, please let me know!
- `program`: Program the Voter2's executable flash with the version in the download flash. This will involve two reboots, one to initiate the update, then a second to boot up with the new version. Note that to be able to program, a download must have been performed since the last boot—and not erased! The entire process should take less than 90 seconds. If your console connection is Telnet, the connection will be lost due to the reboots.
- `xmodem`: Download flash image using XMODEM-CRC protocol. This is early lightly tested code, so requesting feedback. It has been verified with Tera Term only and only works over UART, not Telnet. Telnet is in USASCII mode and does not handle the binary XMODEM protocol. This may be fixable in the future. It only supports downloading too, not verifying.

- `erase`: Erase the download flash. No real reason to do this normally, it was mainly to assist in code development.
- `Dump [addr] [len]`: Dump a range of download flash memory. The address and length are both hex parameters, up to 8 hex characters. There is no real address checking. Also note that for this command the address is the native flash address, not where the address is mapped to in STM32 address space, so 0-4M for the Voter2, 0-32M for the S-Com.

6.16 utest (utest)

This command does a quick check of the two externally facing UARTs and is intended for production testing only. The two UARTs are the Ser2Net UART and the external UART intended for future S-COM variants. These are loopback tests so assume a loopback connector plugged into the UART connection. There is one parameter, the UART to be tested

- `ext`: Test the external S-COM UART
- `s2n`: Test the Ser2Net UART. Importantly, no active Ser2Net connection should be active.

7 APPENDIX A – FACTORY DEFAULT SETTINGS

Listed below are the factory default settings by group:

GENERAL SETTINGS	LOCAL MODE SETTINGS
mystaticip 192.168.2.226	local mode repeater
mystaticsn 255.255.255.0	local cwspd 20
mystaticgw 192.168.2.1	local plfreq 88.5
mystaticdns 192.168.2.1	local deemph on
tftpaddr 192.168.221.200	local precw 500
gpsbaud 115200	local postcw 500
gpsedge rising	local idtime 600
guestpwd password	local hangtime 1500
rootpwd guru	local txdly 100
myname (none)	local pllevel 3
myipmode dhcp	local id N0CALL/R
s2nport 2000	local proc L
s2nptime 100	
rxmode cor	
rssimode analog	
ALLSTARLINK SETTINGS	
dial ipmode static	
dial compression mulaw	
dial packettiming general	
dial unauthrate 1000	
dial port 1667	
dial txdly 100000000	
dial staticip 192.168.2.208	
dial voterpwd ASL3Vote	
dial hostpwd Master_Control	
dial fqdn n0call.dyndns.net	

8 APPENDIX B – SER2NET SETUP

Editors Note 1: The instructions below are *a* method that works, but I'm quite confident it is far from the best way of making things work. There are many other RFC2217 UART-to-Telnet drivers out there and there may be some that are easier to set up and maybe even stay resident instead of the method below. My initial goal was getting *something* to work in order to verify my RFC2217 implementation with the real Motorola RSS software and the process below does that. I'm happy to receive and include better implementations.

Editors Note 2: I took notes, but not the best notes while iteratively figuring out the installation and test process. Now that the software is installed and running, I fear that the instructions may be missing some steps. If you are doing a fresh install and find out something is missing, please let me know.

The hardest part of course is finding a computer old enough to still have a 32-bit processor as that is the only CPU the Motorola RSS software will run on. It will most likely be running Windows XP. Luckily, a computer that old likely still had a serial port and you've been running the Motorola software using the standard UART interface.

The port redirection software recommended to me is com0com and hub4com. A place to download these is on sourceforge at:
https://sourceforge.net/projects/com0com/files/

Select hub4com, the 2.1.0.0 folder and download
hub4com-2.1.0.0-386.zip
Open this ZIP file and save the contents to a directory of your choosing.

Back at sourceforge, go back to the top then select com0com, the 3.0.0.0 folder and download the signed image:
com0com-3.0.0.0-i386-and-x64-signed.zip

That ZIP file contains two images, the x86 and x64 images.
Extract and run the x86 image.
Accept the defaults. Note the install process takes a while!
You will get a "Found New Hardware" popup

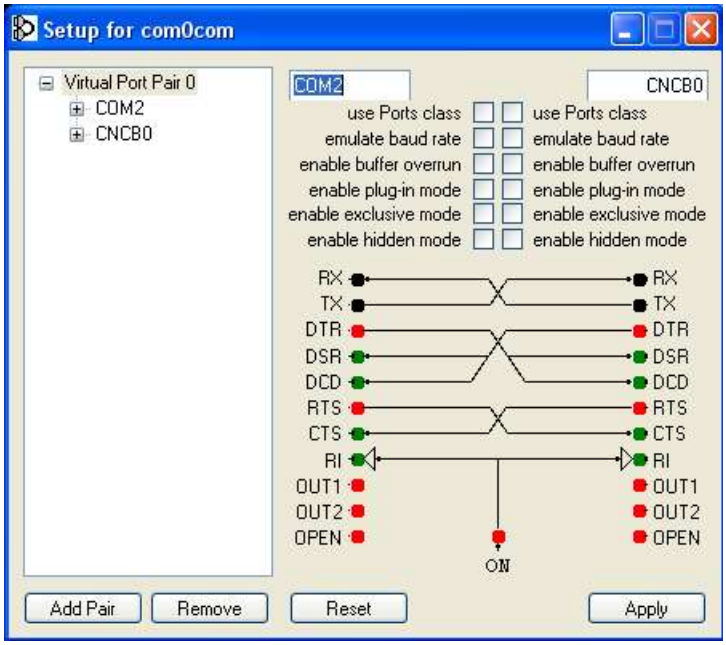
- Say "let Windows do the update automatically" (it works!)
- This happens four times.

Next, run com0com setup. This can be automatic at the end of the install or you can run it manually afterwards, it shows up as an application in the Windows start|all programs|com0com menu.

This brings up the graphical configuration screen for com0com. The default setup has two Virtual Port pairs. You only need one, so OK to delete Virtual Port pair 1 and work with Virtual Port pair 0.

For Virtual Port Pair 0, change the left port to an unused COM port on your computer. Note that the Motorola software only supports COM ports 1-4, so it must be one of those

Leave the right port as CNBC0. Leave all of the check boxes un-checked. In the end, the setup screen should look like this:



Click Apply and close.

Make sure your MTR-2000 and Voter2 are powered up and connected to the same network as the laptop. Note the Voter2's IP address (`show myactualip`).

Go to the folder you installed the hub4com files in to and open a command console in that directory. There should be a batch file called com2tcp-rfc2217.bat.
Enter the DOS command similar to below

```
com2tcp-rfc2217 \\.\CNCB0 192.168.2.163 2000
```

where 192.168.2.163 is instead your Voter2's IP address and if you changed the Voter2's Ser2Net port to something other than 2000, that new value.
If all goes well, a bunch of logging text will scroll by and will pause after a few seconds without any errors, *not* returning to the command prompt. The text is logging information regarding RFC2218 commands.

Leave that window alone and fire up the Motorola RSS software.
Select the COM port you chose during the com0com setup, (say a short prayer) and click on the codeplug download button. Hopefully it works!

After you have finished and closed the MTR-2000 software, just close the command box you ran the com2tcp batch file in.

Whenever you want to run the Motorola RSS software again, you don't need to re-run the com0com setup, but do need to pick up where you open the command console to the hub4com directory and re-run the com2tcp-rfc2217 batch file.

When I used similar software on my modern computer, I did not need to open the command window and run the com2tcp batch file every time, the settings stayed resident. I still hope this is possible with Windows XP as that would mean just firing up the computer and running the Motorola RSS software. I haven't figured this out yet.

9 APPENDIX C – TFTPUPDATE SERVER SETUP

TFTP support for the Voter2 is not fully standard—yet. A special tweak is necessary to the TFTP server setup to work with the Voter2. Why is noted later in this appendix, but the practical requirement is the "randomly" assigned download port called out in the TFTP RFP needs to be a fixed port for the Voter2.

Example TFTP server setup instructions are noted below for Debian and Windows. I am open to including setup instructions for other platforms or applications, but between these two examples you should be able to set up another one if you wish.

After doing the TFTP server setup, continue with Section 7.3 to do the TFTP server file setup.

9.1 DEBIAN tftp-hpa

This is the most likely setup for any Voter2 network, you will have a Debian based server somewhere accessible by all the Voter2s. First install tftp_hpa. Most likely, the command is `sudo apt-get install tftpd-hpa`.

In the file `/etc/default/tftpd-hpa`, you will want the following information:

```
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/srv/tftp"
TFTP_ADDRESS="(your servers IP address):69"
TFTP_OPTIONS="-4 --secure -vvv --create --port-range
30000:30000"
```

(note the last two lines are one line, it gets wrapped in this document)

This should set up the server and the download files will be in `/srv/tftp`.

9.2 WINDOWS OpenTFTPServer

For development purposes, the TFTP application chosen was OpenTFTPServer for Windows:
https://sourceforge.net/projects/tftp-server/
For that project I used the 64-bit multi-threaded version, but fairly confident the other versions will work too. If you get a different TFTP server working, particularly for a different OS, please let me know and I will add it to this appendix.

Installation is pretty straightforward other than the install directory is in c:\. I opted for setting up the TFTP server to load automatically on boot, you may not want to do so. By default, the home directory for downloads is also `c:\OpenTFTPServer\.`

Three parameters in the `OpenTFTPServerMT.ini` file needs changing from defaults.

- `port-range`: Remove the tick and set to `30000-30000`
- `write`: Remove the tick and set to `Y`
- `overwrite`: Remove the tick and set to `Y`

The `port-range` fix fixes the download port range to just that port number. The `write` and `overwrite` fixes allow the `settings backup` command to save data on your computer, the default is disabled. Note that a tick mark is also a comment, so all the parameters preceded by a tick are not actually parameters!

That is all that is needed to set up the TFTP server itself, next is to set up the files that the Voter2 is expecting to see which will be in `c:\OpenTFTPServer\`

9.3 DOWNLOAD FILE SETUP (COMMON)

The first file is the download information file. For the Voter2-2K it is `Voter2-2K.txt`. As other hardware variations of the Voter2 get released (such as the Voter2-SA for the standalone version), they will have a config file name matching the product name. For reference, this name is noted in the source `options.h` file and reported when you log into the console.

The information file contains a single line with 4 space-separated parameters as in this example:

```
0.00C 2025-06-22 Voter2B2-0.00C.srec 4D73
```

The first parameter is the firmware version. This should match the version in the `options.h` file for the build. By convention, the letter at the end will be C for candidate releases for that version and then R for the final release of that version. After the version is the date. Again this is noted in the `options.h` file and noted just for reference. The third parameter is the actual filename of the download. Finally (and not implemented) was going to be a CRC32 of the download file. Alas TFTP modifies text files on the fly, playing with CRs and LFs, so a pure CRC isn't possible.

This setup should allow this single TFTP server to support multiple Voter2 versions and always present the latest version to the Voter2's `update check` command.

Do note also that if you are doing a one-off download you can directly specify the file to download in the `update download` command. This will bypass the settings in the

`download check` command. Be careful as the wrong S-Record file can brick a Voter2.

9.4 REASON FOR A FIXED DOWNLOAD PORT

I hope this "fixed port" work-around is temporary, but the cause is a fairly fundamental disconnect between the TFTP RFP and the way the LwIP drivers work. The issue, to me is the TFTP RFP, it seems very non-standard!

The issue is that while the client (Voter2 in this case) contacts the server at a standard port from its own "high numbered" port, when the server responds, it does not respond from its standard port, but instead from its own random "high numbered" port. Pretty fundamental to IP traffic is that the server responds back to the client using its standard port it got the request from. That way the client knows the response that came back was in response to its request. This is absolutely required for TCP traffic, but evidently optional for UDP traffic. The LwIP netconn drivers (the ones I am using) make that inherent assumption the reply is from the same port. When the TFTP server responds on a different port than the request was sent to, it drops the response assuming it was for some other connection.

The work-around is to fix the TFTP servers "random" address to a specific one, then I hack into the LwIP context structure to change the servers standard port number to 30000 so it will accept the return packet. Not great, but it works.

The assumption on preserving the sever port number is built into the LwIP netconn drivers. The only well documented fix is to use the raw drivers instead. The characteristic of the raw drivers is the LWIP stack just gives all incoming packets to the application and lets the application sort them out. This would then apply to all IP traffic for the Voter2, not just the TFTP traffic. I am using the regular netconn drivers for all the other IP traffic on the Voter2, so not a great option. There is a netconn connection option type called `netconn_raw` noted in the LwIP APIs, but I cannot find any documentation on it. I hope this will be the long term solution.

10 APPENDIX D – AUDIO LEVEL CALIBRATION

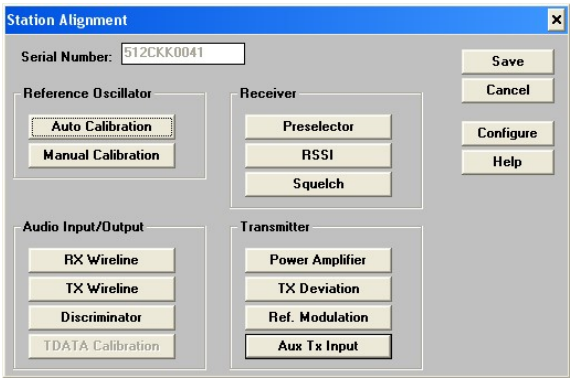
Audio level calibration must be performed on any new repeater/Voter2 setup to ensure a consistent audio level across all repeaters in the network. This must be performed for both repeater receive audio as well as repeater transmit audio. The Voter2 does not have any audio gain adjustments of its own but instead relies on the input/output level controls built into the MTR-2000.

This calibration process is similar to the process for the Voter/RTCM and targets the same levels as well.

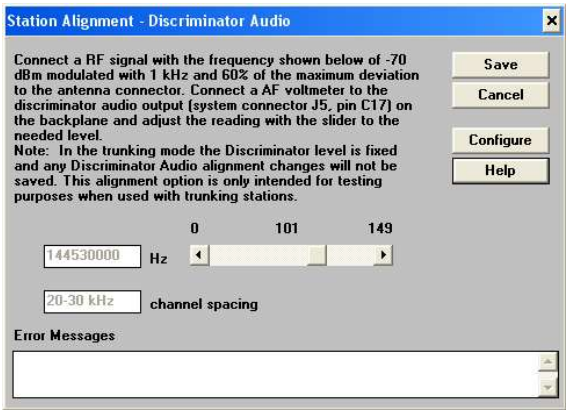
To perform this calibration, you will need a service monitor capable of generating a 1kHz reference tone with a 3kHz deviation and without a CTCSS tone. It should also be able to analyze an incoming RF signal and determine the frequency and deviation of that signal.

The procedure is as follows:

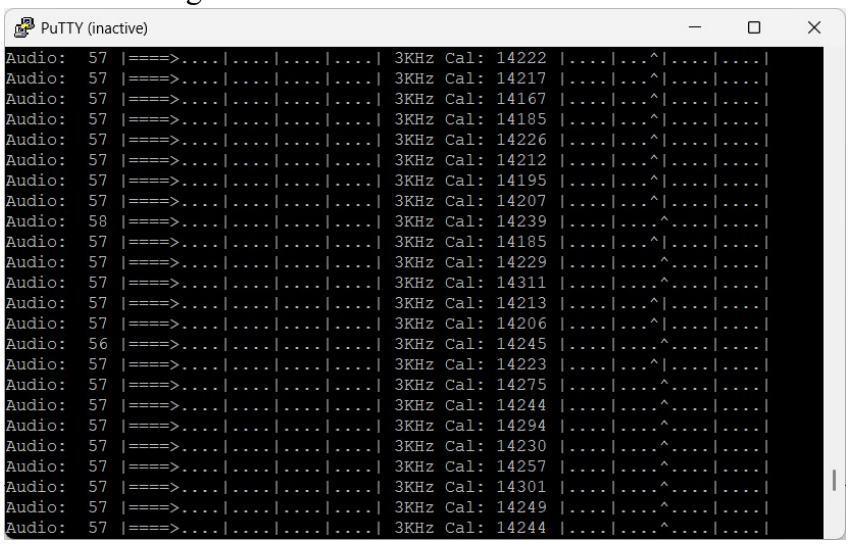
1) Start up the Motorola RSS software and download the codeplug for your repeater.
2) Highlight the downloaded codeplug window (important!), then select Service|Station Alignment. You should get this menu:



3) Click the Discriminator button and you will get this:

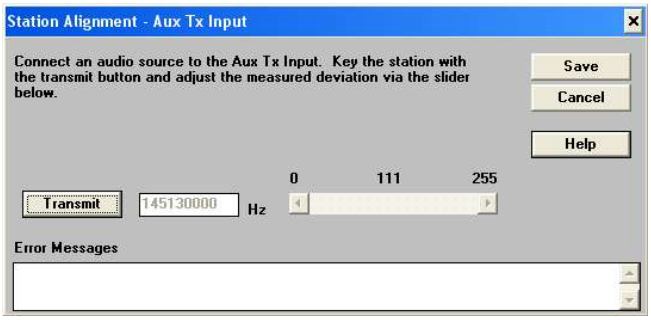


4) With the service monitor feeding the 1kHz/3kHz deviation tone, enter the CLI command `status level`. The left bar is overall audio level and OK to ignore for calibration. The right bar is the audio level "zoomed in" on where the level needs to be to be calibrated. If the audio level is well below the calibration point you will see "<<" on the left, if well above ">>" on the right, and if in range a "^" showing the level. Adjust the slider button on the RSS window to get the caret over the middle line. The target value is 14,328 which is the peak-peak value of the uncompressed audio. The value has noise, so the graph makes it easier to center. The MTR value should be between 95 and 110. Note that at least with my setup there is some drift that oscillates. Center as best you can so the drift is equidistant to either side of the center line. When complete, your screen should look something like this.



5) When good, click on save in the MTR-2000 Discriminator box and save the codeplug.
6) For Aux Tx Input calibration, make sure the Voter2 is in local mode and that no CTCSS tone generation is enabled. In this mode the Voter2 will echo exactly what is coming in. This should be the just-calibrated input you obtained in step 4.

7) On the RSS software, back in the alignment window in step 2, click on the Aux Tx Input button and you should get this window:



8) Click on the Transmit button to enable transmitting and observe the measurements on the service monitor. The tone had better be 1kHz! Adjust the slider in the RSS window to get a deviation as close as you can to 3kHz. The value should be in the 105-115 range. Once calibrated, click on save and then save the codeplug

You have completed audio levels calibration!

11 APPENDIX E - RSSI AND SQUELCH CALIBRATION

Important: RSSI calibration must be performed after audio level calibration!

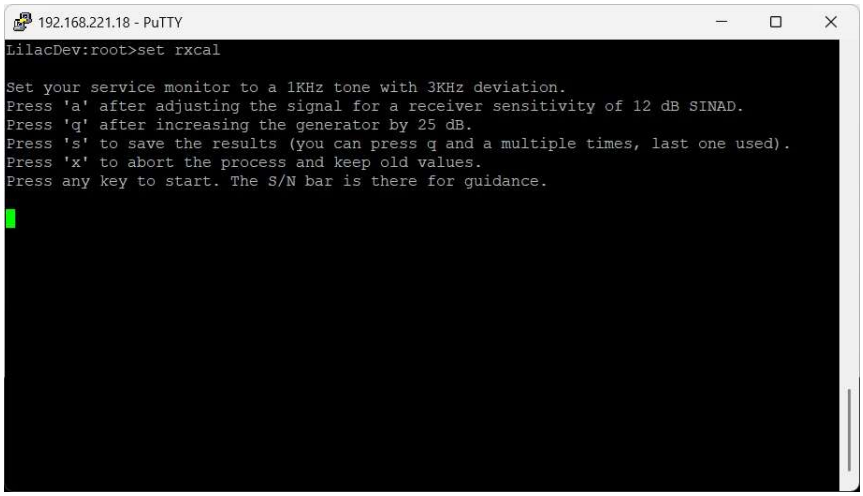
Currently calibration is only supported for analog (vs HPF/BEW modes) RSSI. HPF/BEW RSSI levels are hard coded. See the `set rssimode` command documentation for details on these modes. This may change in the future.

The test setup for analog RSSI calibration can be the same as for audio level calibration, a service monitor generating a 1kHz tone with 3kHz deviation with the ability to vary the RF signal strength. The tone isn't required, but handy if you want to hear noise vs signal. Similarly, the S/N meter displayed by the Voter2 once you start the calibration process assumes the tone, so if you skip the tone, ignore the S/N meter.

If you want to actually measure SINAD, the place to monitor the Rx signal is P7 (labeled RXAUD), S is signal, G is ground. This signal is present regardless of the MTRs COR state.

If you want to hear the audio on a radio it may be handy to unconditionally open the squelch so that you can hear the Rx audio regardless of the MTRs COR state. The command to do this is `set rxmode on`.

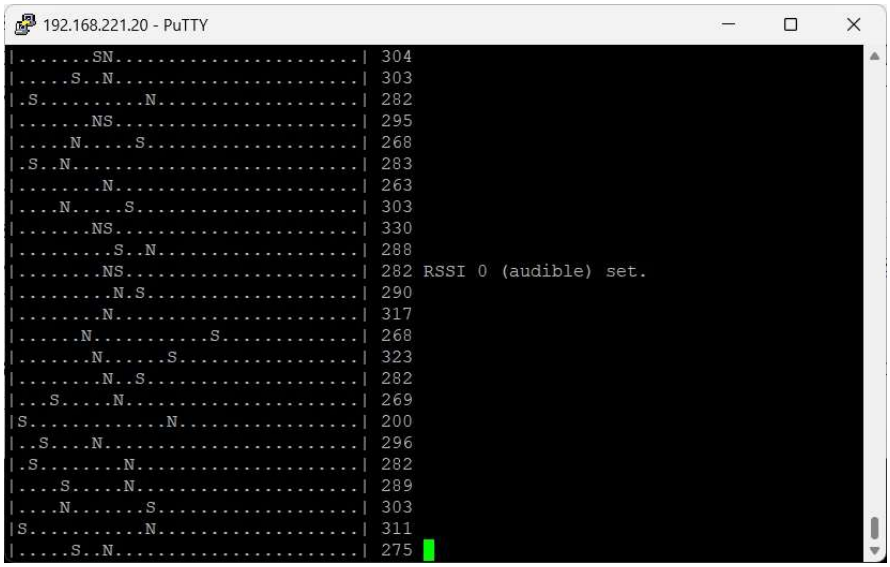
The command to start calibrating is `set rxcal`. You will be shown a quick summary of the procedure and commands as shown below before pressing any key to start the process.



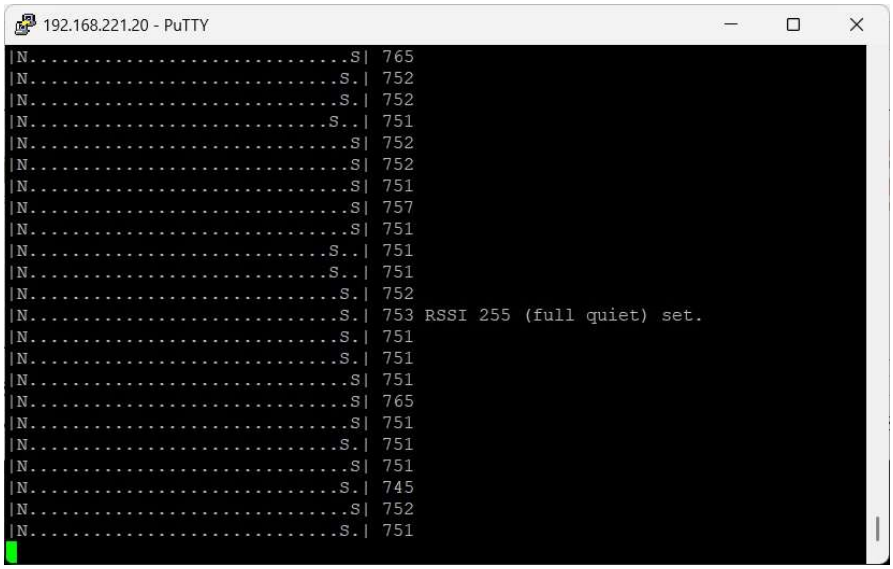
After pressing any key to start you will be shown a waterfall graph showing the relative amplitudes of the 1kHz reference tone (S in the graph—assuming you have the tone) and

all other frequencies (N in the graph) to get an idea of signal and noise strength. This is a pretty arbitrary graph, what matters is what you will be hearing being transmitted.

First adjust the signal strength on the service monitor to get a sensitivity of 12dB SINAD. This is the level that will be considered the 0 RSSI value the Voter2 will report, so the lowest acceptable level. Please make note of this service monitor level as you will come back to it for the squelch calibration you will be doing in Appendix E. Once at that level, press 'a' to save that RSSI. You will see a note scroll by saying the setting has been noted. A, by the way is for readable audio.

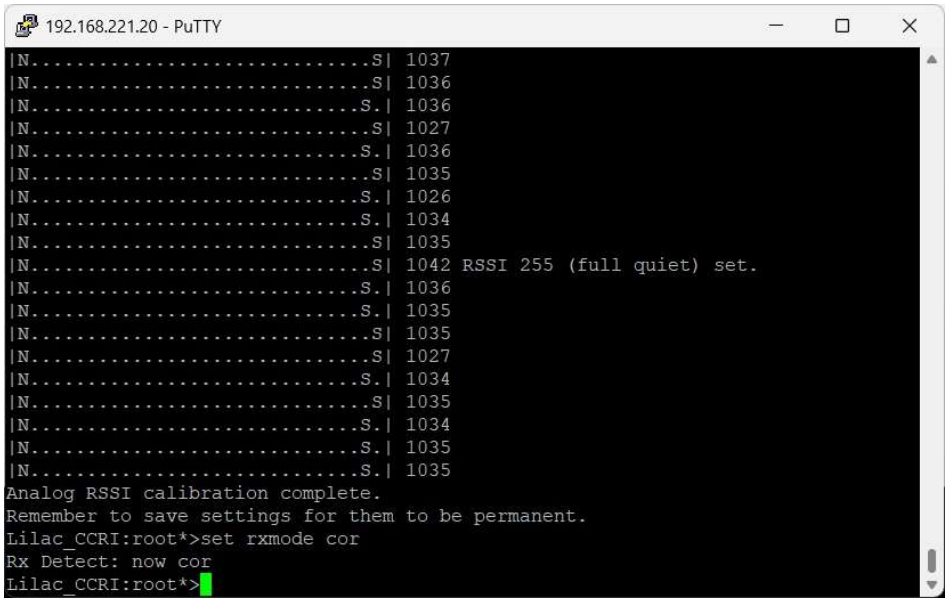


Then, increase the signal level on the service monitor by 25dB, this should now be full quieting. This value is important as it will most closely match what the original Voter reports as full signal (an RSSI of 255). Once at this level, press 'q' to save that RSSI. You will see a note scroll by saying the setting has been noted. Q, by the way is for full quieting.



You can press ‘a’ and ‘q’ as many times as you need during this process, only the last two will be used when you press ‘s’.

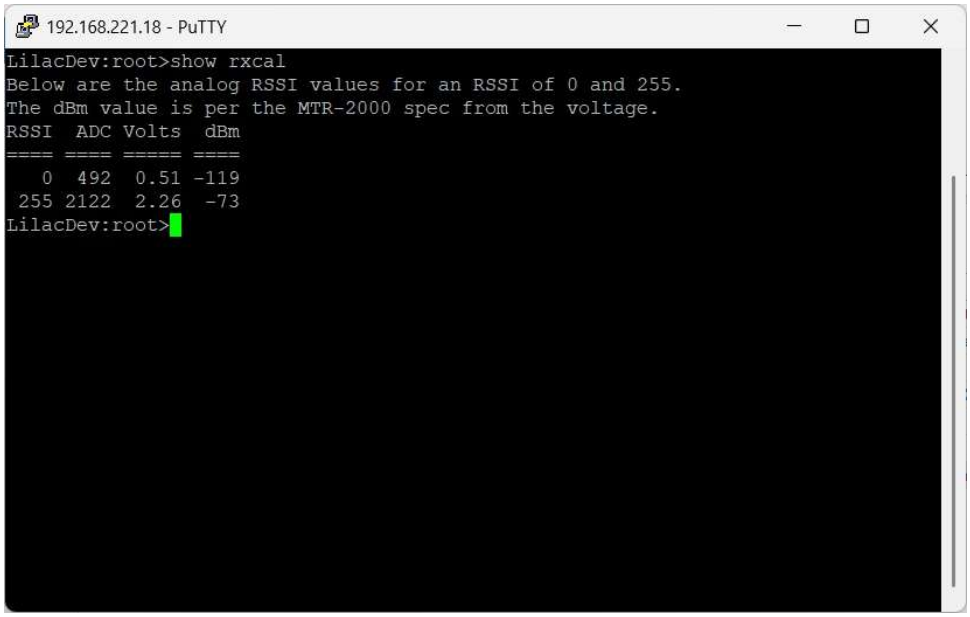
If you are happy with both settings, press ‘s’ to save them and make them active. Note that while they are immediately active, these settings are only stored in volatile memory, not permanent NVM. This means you can test and verify you like the settings before making them permanent. To save them in NVM, enter `settings save`.



If you want to abort the process without making any changes, press ‘x’ to exit.

If you set the Rx squelch to open, don't forget to set it back! :)

If you want to see the results of the Rx RSSI calibration you can enter the command `show rxcal`. Your results will look something like this:



Values are shown for the two "0" and "255" RSSI calibration points you identified. The ADC value is more for debugging, the Volts value is the MTR-2000 analog RSSI value (before the voltage divider, so actual MTR voltage), and finally the dBm value based on the formula in the MTR manual to derive dBm from the voltage.

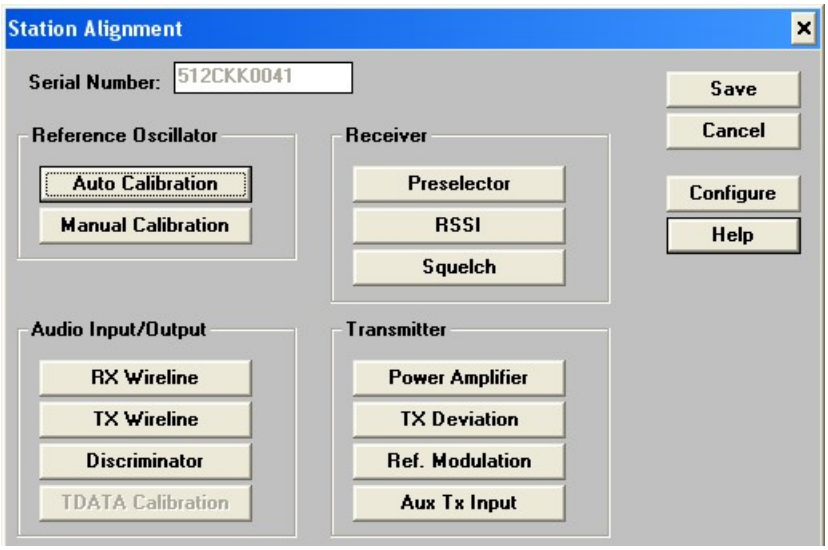
12 APPENDIX F - SQUELCH CALIBRATION

A key difference between the Voter2-2K and Voter is that the Voter2 makes use of the MTR-2000s squelch circuit. It's there, it's good, so taking advantage of it--at least if you configure the Voter2 to use it. The setting is `rxmode`. With that command you can tell the Voter2 to use either COR (squelch), CTCSS, or both as the trigger to retransmit audio. Hence for this test you should set `rxmode` to `cor`.

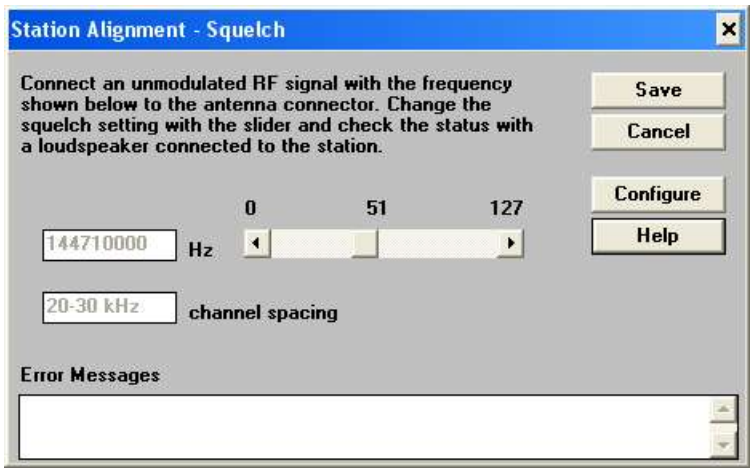
The same service monitor setup in Appendixes D and E is applicable here too. While the 1kHz tone at 3kHz deviation isn't necessary for this calibration, it doesn't hurt and at least for monitoring the test is handy.

Set the service monitor signal level back to the "0 RSSI" setting you determined in Appendix E to get 12dB SINAD. If you forgot to note that level, just re-do that part of the test to get it again.

Fire up the Motorola RSS software, download the codeplug, make sure your downloaded codeplug is highlighted and select the Service|Station Alignment menu. It should look like this:



Click on Squelch. It should look like this:



Adjust the slider such that the MTR-2000 breaks squelch at that level. If it doesn't break squelch, you may have the `rxmode` set wrong, likely to `corctcss` and not have a tone.

If you would like to see the Voter2s perspective on the calibrating, you can enter the command `status rssi`. Of note the last two dots at the end represent the MTR-2000s COR and CTCSS status signals.

Once set, click on save, then on save again. This causes the new squelch value to be loaded into your MTR2000 and forces a reset.

13 RELEASE NOTES

During initial releases I was not tracking changes—there were too many! :) I am starting release notes effective the Rev 0.04 release

Release 0.04:
- Changed factory default RSSI to HPF. In real life it works better than analog
- Added interpolation tables for analog and HPF RSSI values to match Voters
- Added filename override for settings backup/restore command
- Added filename override for update download command

Release 0.05:
- Revisited "regular" audio level bargraph. It seemed low, but is by design this way.
- Added COR and RDSTAT (CTCSS) states to the status RSSI command (RDSTAT theoretical)
- Added block number checking to settings backup/restore
- Added a spinny thing to show life during flash erase for update download and update erase commands
- Doing some limited (3-packet) HPF RSSI averaging to stabilize it a bit.

Release 0.06:
- Improved (decreased) CPU loading some more during flash programming.
- Voice filter was 400Hz-2.4kHz. Dave/Doug said it should be 400Hz-4kHz. I can get 475Hz-4kHz.
- Found a bug where FLATAUDIO bit in Dial Byte wasn't being interpreted correctly.
- DSP/BEW code theoretically works, needs real-world calibrating
- Rx RSSI DSP routines now only execute if that RSSI mode is activated (or diags running).
- Re-measured (and documented) execution times of all of the major Rx DSP routines
- Removed conditional support for non-CMSIS DSP routines (CMSIS-only now)
- Added 'show ethernet' to show all Voter2 Ethernet settings with one command.
- Other readability enhancements per Dave's requests.
- Added a status 10mhz command to show the local clock frequency based on GPS1PPS.
- Added an RSSI calibration procedure for analog RSSI.
- CLI commands are now case-insensitive
- Early OCVCXO support (hardware not released yet)

Release 0.07:
- I missed a use case in CLI case insensitivity support where ASL passwords were being converted to all lower case.
- No manual changes, this release was for a bug fix.

Release 0.08:

- Added support for last-command recall (up to 10 commands). Only editing support is backspace, not character inserting/deleting within a line.
- Fixed a bug that could lock the Voter2 if GPS logging enabled at the IO level.
- Manual change only: Added an index at the start of this manual.
- Manual change only: Moved Rxtest to the command area, it is not a setting
- Manual change only: Packettiming description error, `general` vs `gp` for general purpose mode.
- Manual change only: Added a section after scope to list all other manuals for the Voter2.

Release 0.09:

- Added a user settable GPS time offset (numeric and parametric), mainly for interop with older GPSes. Only one parametric offset so far, `thunderbolt (thu).`
- Changed the factory default for dial timing from general purpose to GPS-timed.
- Found/fixed a bug that was sending GPS ASL packets with a "0" timestamp
- Made the challenge string a random number assigned at boot (like the Voter) and not a user setting.
- Clarified Host/Client password descriptions in the UI (thanks Kevin)
- Reformatted GPS lat/long strings in `status gps` to separate degrees from minutes (Dave request)
- Added reminder to run `update program` after `update download` in firmware update sequence
- Adjusted the "`rxtest dev3k`" deviation to the right value (it was a placeholder I forgot to get back to)
- Fixed `rssimode`, `rxtest` and `update help` texts, they were out of date
- Cleaned up Sat Signal for `status gps` a bit, fewer blank lines.
- XMODEM download support! `update xmodem.` Currently download support only, not verify and only UART only for now. XMODEM over Telnet is messy.
- Added `status version` for HW and SW versions, deleted `hwrev` from `show` as it is now redundant.
- CTCSS parameters are now global and independently enablable for local, general purpose and GPS modes
- renamed `xuart` command to `utest` with parameters `[ext|s2n]` to test either s2n or external UART.
- Found bug that hung Ethernet on cable unplugging. Also now re-run DHCP on cable reconnect.
- Manual change only: Included all the shortcuts for all commands

- Manual change only: Clarified host/client password descriptions.
- Manual change only: Explained sync issues for `status gps`.